State management in ASP.NET Core MVC

🔷 1. Why State Management is Needed

In real-world applications, you must retain data such as:

- Logged-in user information
- Shopping cart items
- User preferences
- Temporary messages between requests

Without state management:

- Every request starts from scratch
- No continuity in user experience

🔷 2. Types of State Management

State management in ASP.NET Core MVC is broadly classified into:

✅ A. Client-Side State Management

Stored in the **user's browser**

1. Cookies

- Small key-value data stored in browser
- Sent with every request

Response.Cookies.Append("UserName", "San");

var name = Request.Cookies["UserName"];

✓ Characteristics:

- Persistent (can set expiry)
- Limited size (~4KB)
- Less secure (can be modified)

---

2. Query Strings

- Data passed in URL

Example:

https://example.com/Home/Index?id=10

public IActionResult Index(int id)

✓ **Characteristics:**

- Simple and visible
- Not secure
- Length limitations

---

3. Hidden Fields

Used in forms to store data

<input type="hidden" name="UserId" value="101" />

✓ **Characteristics:**

- Works only with form submissions
- Can be tampered

---

4. TempData (Hybrid but often client-backed)

- Used to pass data between requests (especially redirects)

TempData["Message"] = "Saved Successfully";

var msg = TempData["Message"];

✓ **Internally:**

- Uses **cookies or session**

✅ **B. Server-Side State Management**

Stored on the **server**

1. Session State

- Stores user data per session

HttpContext.Session.SetString("UserName", "San");

var name = HttpContext.Session.GetString("UserName");

✓ Characteristics:

- Stored on server
- More secure than cookies
- Requires configuration

🔧 **Setup in ASP.NET Core:**

builder.Services.AddSession();

app.UseSession();

---

2. TempData (Server-backed option)

- When configured with session provider

---

3. Cache (Application-level state)

a) In-Memory Cache

IMemoryCache cache;

cache.Set("key", value);

b) Distributed Cache (Redis, SQL Server)

- Used in scalable apps

---

4. Database

- Persistent storage (long-term state)

Example:

- User profile

- Orders
- Settings

🔷 3. TempData vs ViewData vs ViewBag

Feature	**ViewData**	**ViewBag**	**TempData**
Type	Dictionary	Dynamic	Dictionary
Lifetime	Same request	Same request	Across requests
Use Case	Pass data to View	Same as ViewData	Redirect scenarios

---

🔷 4. Session vs Cookies

Feature	**Session**	**Cookies**
Storage	Server	Client
Security	High	Low
Size Limit	Large	~4KB
Performance	Slower (server hit)	Faster

🔷 5. When to Use What (Best Practices)

Scenario	Recommended Option
Authentication	Session / Claims
Small persistent data	Cookies
Redirect messages	TempData
Large data	Database
High performance shared data	Cache

🔷 6. Key Concepts to Remember

- HTTP is stateless → State management is required
- Choose **client vs server storage** based on:
    - Security
    - Data size
    - Performance
- Avoid storing sensitive data in cookies
- Use **Session + Cache + DB** for scalable apps

🔷 7. Real-Time Example (Flow)

1. User logs in
2. Store user ID in Session
3. Use it across requests
4. Show welcome message using TempData
5. Persist user data in Database
